A Bridging Model for Parallel Computation

Valiant (1990): Summary

Leslie G. Valiant. *Communications of the ACM* 33(8), 103–111, 1990.

Motivation

A bridging model gives hardware and software a shared target. As the von Neumann model does for sequential computing, Valiant's proposal aims to support portable parallel programs across different physical machines. The abstraction must allow both efficient compilation and efficient hardware implementation. An *optimal simulation* loses only a constant factor, independent of processor count, rather than a growing logarithmic factor. BSP is an intermediate model between languages and hardware, not a particular language or network topology.

The **bulk-synchronous parallel (BSP)** model combines:

1. Components performing processing and/or memory functions, typically processors attached to local memory.
2. A communication network for delivering point-to-point messages between the various components.
3. Barrier synchronization: a hardware or software mechanism to synchronize all or a participating subset of components at regular, programmable time intervals, described by the parameter L.

A computation consists of **supersteps** combining local computation and message transmission and arrival. Local operations use data available before the superstep starts. Completion is checked every L time units; unfinished work receives another interval, and the next superstep begins only after all participating components finish. No message-arrival order is assumed. Replication, broadcasting, and combining are not free router primitives.

The main machine parameters are the physical processor count p, the synchronization periodicity L, and the relative communication cost g. An *h-relation*, in which each component sends and receives at most h messages, costs gh time units for sufficiently large h to amortize startup costs. Smaller g means higher communication throughput relative to local computation. Hardware limits how small L can be; larger L requires more independent work between checks. Independent components may disable synchronization to avoid unnecessary waiting.

Parallel slackness

Programs expose v virtual processors, significantly more than the p physical processors, for example

$$v = p \log p.$$

The compiler maps multiple virtual processors to each physical processor to schedule useful work, pipeline communication, and absorb uneven access loads. Slackness is excess available parallelism, not idle processors or redundant computation. The programmer must expose independent instruction streams; arbitrary sequential code is not assumed to become parallel automatically.

Memory management and concurrency

Balanced arithmetic is insufficient if memory requests overload a few components.

Hashing for automatic allocation.
Pseudo-random hashing maps symbolic addresses to distributed physical storage. With only p distinct requests randomly assigned to p components, the maximum load can be about $\log p / \log \log p$, despite a mean of one. With $p \log p$ requests, the maximum is within a constant factor of the $\log p$ mean with high probability. Batching enough independent requests therefore absorbs imbalance efficiently. For constant g, suitable L, and sufficient slack, a step of v virtual processors accessing distinct locations can be simulated in $O(v/p)$ time with high probability. Hashing and context-switching costs still affect practical constants.

Simultaneous accesses (CRCW PRAM).
With $g = O(1)$, $L \geq \log p$, and $v = p^{1+\varepsilon}$ for fixed $\varepsilon > 0$, the paper gives a general concurrent-read/concurrent-write PRAM simulation based on parallel integer sorting. It takes $O(v/p)$ time per simulated step in a suitable BSP configuration, with constants that worsen as ε approaches zero. Synchronization must be compatible with the work available to amortize it. Replication and combining are charged as operations at the components.

The example $v = p \log p$ does not alone meet the general CRCW conditions. If at most c requests target one location and sufficient slack of order $cp \log p$ is available, simpler source replication and destination combining can suffice, with their processing and communication costs included.

Direct BSP algorithms without hashing

Explicit data placement and communication can reduce overhead and required slack, especially when g is large. Costs depend on problem size n and machine parameters p, g, and L.

Matrix multiplication. For the standard product of two $n \times n$ matrices on $p \leq n^2$ processors, assign each processor an $(n/\sqrt{p}) \times (n/\sqrt{p})$ output block. Each performs $O(n^3/p)$ arithmetic operations and receives $O(n^2/\sqrt{p})$ input elements. The computation can retain optimal $O(n^3/p)$ runtime when

$$g = O\left(\frac{n}{\sqrt{p}}\right), \qquad L = O\left(\frac{n^3}{p}\right).$$

These conditions express the balance between useful work and overhead.

Other examples. Tree-based broadcast spreads replication across components; related recursion supports parallel prefix for associative operations. FFT butterfly computations can be grouped into larger local tasks. Columnsort alternates local sorting and regular data permutations, separating computation from communication at a coarse scale.

Efficient implementation in hardware

On a hypercube, one packet may take $O(g \log p)$ time, yet $\log p$ packets per component can be routed in the same asymptotic time through pipelining and randomized routing. Throughput is therefore more important than latency for one isolated packet. With balanced communication hardware and $L > g \log p$, a superstep with L local operations and an L/g-relation can be simulated in $O(L)$ time. Butterfly networks and an optical model provide further implementation examples.

Main contribution, insight, and limits

BSP offers a common interface for languages, algorithms, and hardware through slackness, batched communication, and controllable synchronization. Its efficiency results require enough parallelism, suitable g and L, and balanced communication capacity. Constant-factor optimality also leaves practical constants to evaluate: portability does not imply that every program runs efficiently on every machine.